How-To Library

Informatica

Best Practices for the Blaze Engine

© Copyright Informatica LLC 2017, 2021. Informatica, the Informatica logo, Informatica Big Data Management, and Informatica PowerCenter are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

The Informatica Blaze engine integrates with Apache Hadoop YARN to provide intelligent data pipelining, job partitioning, job recovery, and high performance scaling. This article outlines best practices for designing mappings to run on the Blaze engine. The article also offers recommendations to consider when running mappings on the Blaze engine.

Supported Versions

- Informatica Big Data Management™ 10.0, 10.1, 10.1.1

Table of Contents

Overview. . . . 2
Concepts and Terminology. . . . 3
Configuration Recommendations. . . . 3
Hadoop Cluster Configuration Recommendations. . . . 3
Informatica Domain Configuration Recommendations. . . . 4
Design Recommendations. . . . 5
Generate Unique Sequences with the Sequence Generator Transformation. . . . 5
Set the Number of Rows to Preview for Large Hive Sources. . . . 5
Ensure Data Types and Precision Values Match Within Mappings. . . . 7
Run-time Recommendations. . . . 8
Using Informatica Developer to Run Mappings. . . . 8
Optimize Caching Transformations. . . . 9
Enable Compression for the Distributed Cache. . . . 9
Disable Map-side Aggregation if a Unique Key is Used as the group by Key in an Aggregator Transformation. . . . 10
Enable Partitioning for Data Processor Transformations. . . . 11
Run Mappings with Non-partitioned Relational Database Sources on the Hive Engine. . . . 11
Set Preferred Partition Size for Partitioned Hive Targets. . . . 13
Disable Map-side Join for Large Data Sources. . . . 13
Disable YARN pmem-check if Using Update Strategy Transformation. . . . 14

Overview

Informatica Big Data Management enables your organization to perform big data integration and transformation without writing or maintaining Apache Hadoop code. The Blaze engine complements Informatica's Big Data Management solutions to provide optimal data processing performance.

The Blaze engine is a high-performance execution engine that enables distributed processing on a Hadoop cluster. The Blaze engine is optimized to deliver scalable data processing through leveraging Informatica’s cluster-aware data integration technology. You can run mappings on the Blaze engine to take advantage of the engine's ability to efficiently process large, complex batch workloads.

Review the guidelines on designing mappings to run on the Blaze engine outlined in this article. Also review the run-time recommendations provided to help you maximize performance and avoid potential issues when running mappings on the Blaze engine.

Concepts and Terminology

You must be familiar with big data and Hadoop concepts to benefit from this article.

Ensure that you understand the following Blaze engine terms and concepts used in this article:

- Data Exchange Framework

  The Data Exchange Framework (DEF) is responsible for data shuffle across data nodes on the cluster. The DEF daemon process runs on every data node facilitating data shuffle, the process by which data from the mapper tasks is prepared and moved to the nodes where the reducer tasks run. The DEF is analogous to the Spark shuffle service.

- Tasklet

  Tasklets are Data Transformation machine processes that run within containers. Tasklets are analogous to cores in Spark or mappers in Hive on MapReduce.

- Container

  Containers in the Blaze engine are analogous to Spark executors. By default, each container is configured to run four tasklets and one DEF client.

Configuration Recommendations

When you run a mapping on the Blaze engine, the Data Integration Service pushes the mapping job to nodes in a Hadoop cluster. Configure both the Hadoop environment and the Informatica domain nodes running the Data Integration Service to optimize performance.

Hadoop Cluster Configuration Recommendations

Configure the following Blaze engine settings in the Hadoop environment.

Specify the Blaze engine log directory.

Create a directory under `$HADOOP_NODE_INFA_HOME` on each Hadoop cluster node. Grant all users access to this directory.

For example:

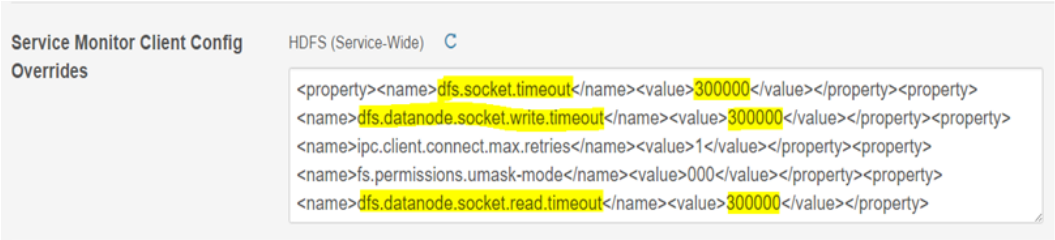
```
/opt/Informatica/blazeLogs
```

Set HDFS access timeout properties.

Mapping execution might fail on large HDFS (Hadoop Distributed File System) queries due to access timeout errors. Set the following HDFS timeout properties on the Hadoop cluster to sufficient values to avoid errors.

- `dfs.client.socket-timeout`
- `dfs.datanode.socket.write.timeout`
- `dfs.datanode.socket.read.timeout`

You can use tools such as Apache Ambari or Cloudera Manager to update the HDFS properties. The following image shows how you can set the properties in Cloudera Manager:



Informatica Domain Configuration Recommendations

Set the following Hadoop configuration properties on every domain node running the Data Integration Service. You set Hadoop configuration properties in the `hadoopEnv.properties` file, which is located in the following directory on each domain node:

`<Informatica installation directory>/services/shared/hadoop/<Hadoop distribution>/infaConf/`

Set memory and tasklet properties.

The amount of memory and number of tasklets available in each container control parallelism in the Blaze engine. The default memory and tasklet configuration should be adequate for most use cases. However you can increase both memory and the number tasklets in each container if needed.

The following table describes the properties to set in the `hadoopEnv.properties` file:

Property	Description
infagrid.def.max.memory	The amount of memory in megabytes allocated to the DEF Daemon. Default is 4096 MB.
infagrid.orch.scheduler.oop.container.pref.memory	The amount of memory in MB allocated to each container running on a node. Default is 5120 MB.
infagrid.orch.scheduler.oop.container.pref.vcore	The number of tasklets that run within a single container when the Blaze engine runs a job. Default is 4.

Configure Oracle relational database table access through native drivers.

The following table describes the properties to set in the `hadoopEnv.properties` file:

Property	Description
infacal.hadoop.env.entry.PATH	Set to the location of the Oracle client libraries.
infapdo.env.entry.oracle_home	Set to the value of the ORACLE_HOME environment variable
infapdo.env.entry.tnsadmin	Set to the value of the TNS_ADMIN environment variable.
infapdo.env.entry.ld_library_path	Set to the value of the LD_LIBRARY_PATH environment variable.

The configuration changes take effect after you recycle the Data Integration Service on each node.

Design Recommendations

You use the Developer tool to create mappings to run on the Blaze engine. Consider the recommendations in this section when you design and develop mappings.

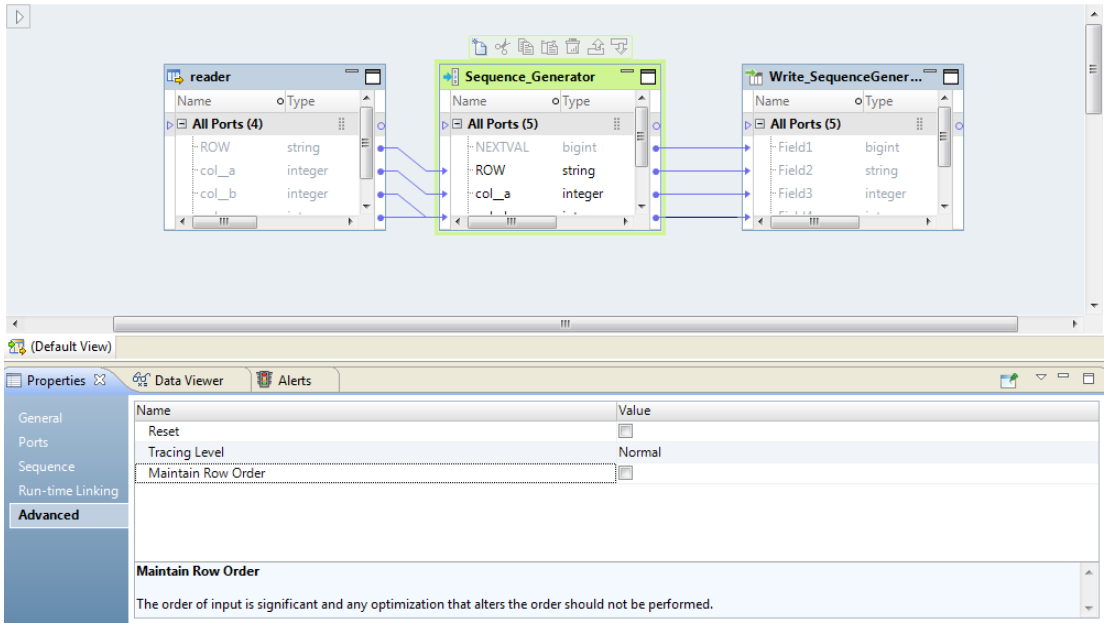
Generate Unique Sequences with the Sequence Generator Transformation

The Sequence Generator transformation is a passive transformation that generates numeric values. The Sequence Generator transformation maintains a global numeric sequence, or generates a unique sequence.

Maintain a global sequence to force a mapping to run within a single partition. Use this option only if the use case requires that the Data Integration Service not perform any optimization that can change the row order.

Generate a unique sequence to allow a mapping to run within multiple partitions. Use this option for better performance.

To generate a unique sequence, select the Sequence Generator transformation in the developer tool, and then de-select **Maintain Row Order** in the **Advanced** properties tab, as shown in the following image:



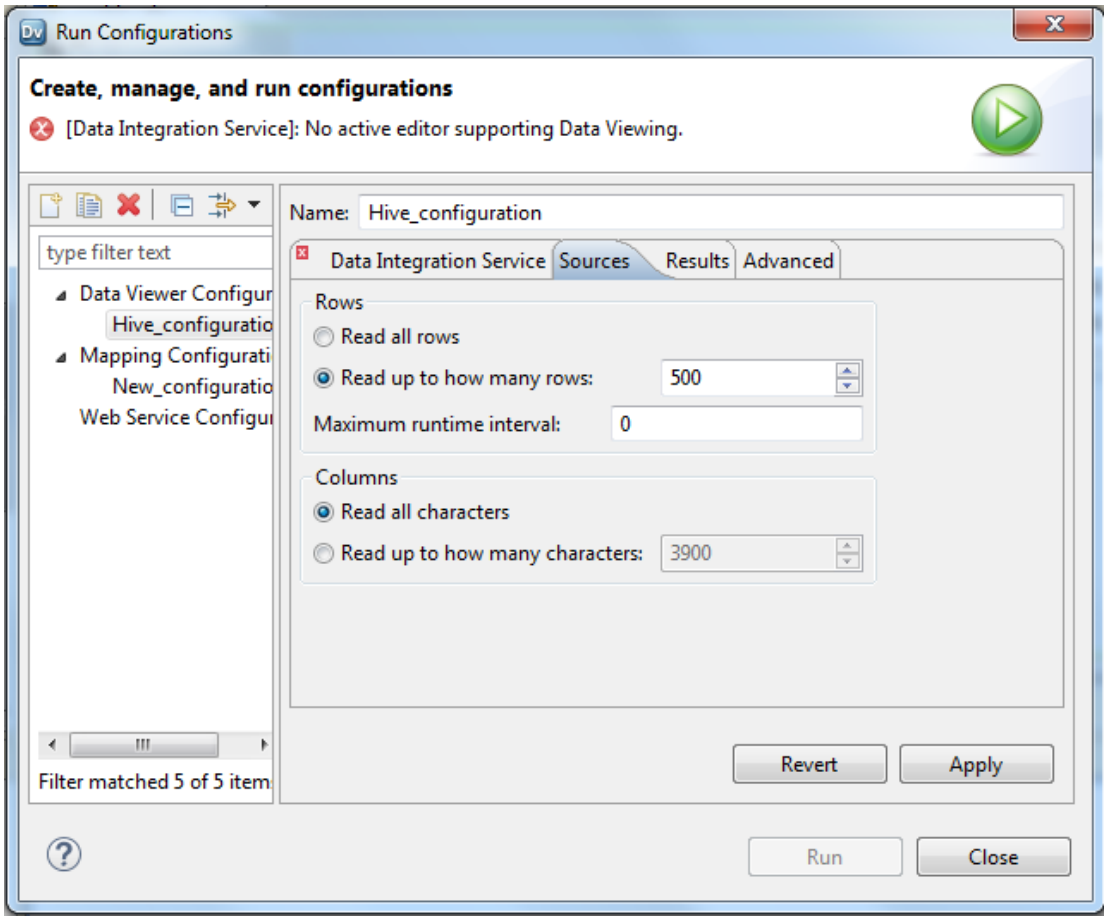
Set the Number of Rows to Preview for Large Hive Sources

You can use the Data Viewer in the Developer tool to preview data in a Hive source included in a mapping. Based on the size of the source file, you might want to explicitly restrict the number of rows in the source to view.

By default, the Data Viewer reads all rows in a source. For large Hive sources, a MapReduce job starts to read all rows in the source. A large data source might take a significant amount of time to process.

For better performance, create multiple run configurations. Create a Data Viewer Configuration, which controls the settings that the Developer tool applies when you preview output in the Data Viewer view. Create a Mapping Configuration, which the Data Integration Service uses to generate an execution plan to run mappings on the Blaze engine.

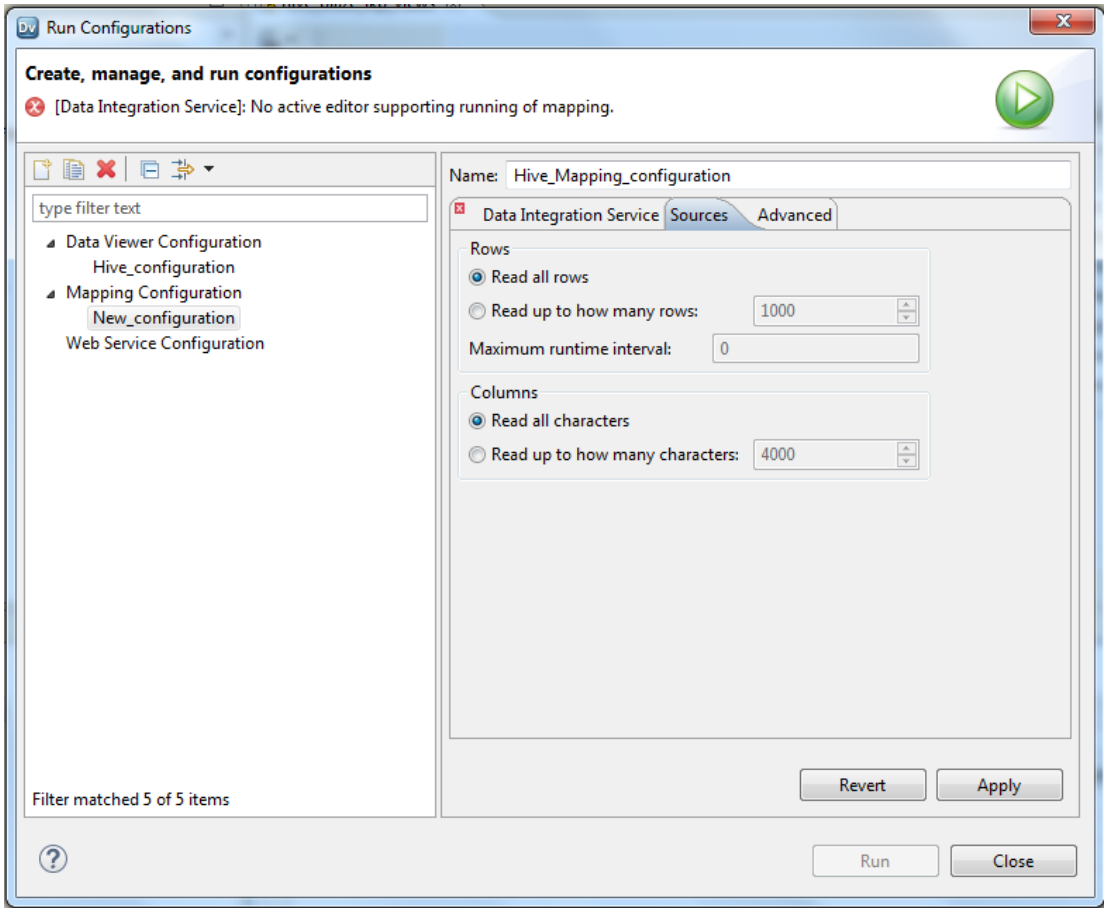
Create a Data Viewer Configuration and explicitly set the number of rows to read, as shown in the following image:



1. Select the mapping in the Developer tool.
2. Select **Run** > **Open Run Dialog**.
3. Click **Data Viewer Configuration** and create a configuration.
4. Click the **Sources** tab.
5. Set the number of rows to read in the **Read up to how many rows** field.

Perform the same steps to create a Mapping Configuration. The Data Integration Service must read all rows in the Hive source to generate the plan.

Create a Mapping Configuration and select the **Read all rows** option in the **Sources** tab, as shown in the following image:



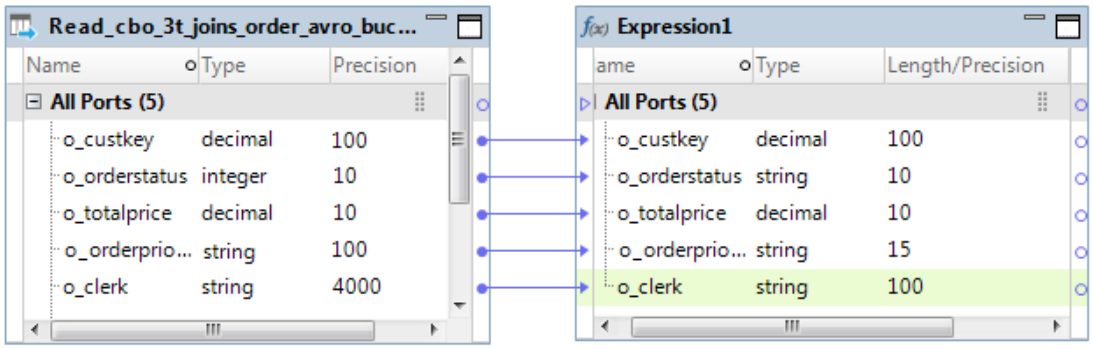
Ensure Data Types and Precision Values Match Within Mappings

Using mismatched data types or precision values can increase computing overhead when processing large files. Ensure properties set in propagating ports are consistent across sources, transformations, and targets within a mapping.

If data conversion is required in a mapping, the Developer tool adds an Expression transformation to the mapping at run-time to perform the conversion, which might result in longer processing time.

If the precision value set in a string port is larger than the actual length of the string, more buffer memory than is actually needed is allocated on the node, which might also impact performance.

The following image shows a mismatch between the data type and precision values in some of the ports in the Read transformation and the corresponding data type and length/precision values in the Expression transformation:



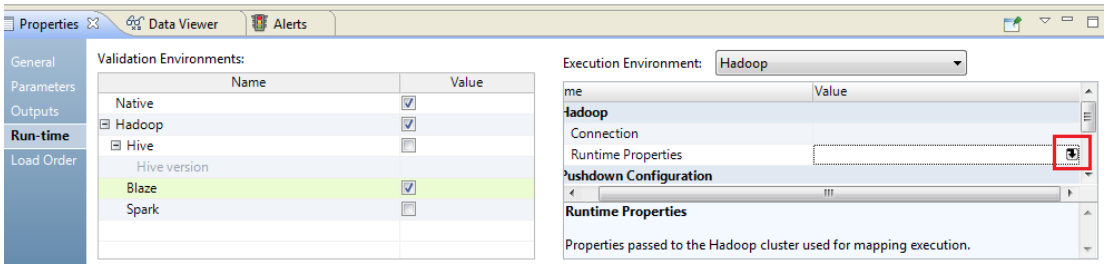
Run-time Recommendations

Configuring mappings to run efficiently on the Blaze engine depends on many factors, such as the size and configuration of your Hadoop environment and the size of the data you process. Review the recommendations provided in this section to determine which recommendations make sense for your specific situation.

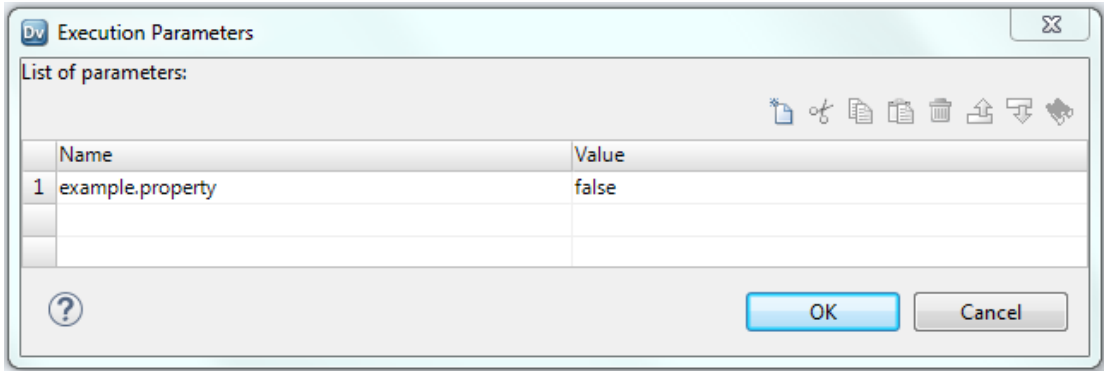
Using Informatica Developer to Run Mappings

You use the Developer tool to run mappings on the Blaze engine. You can also set run-time properties on mappings, and set advanced properties on specific transformations within mappings.

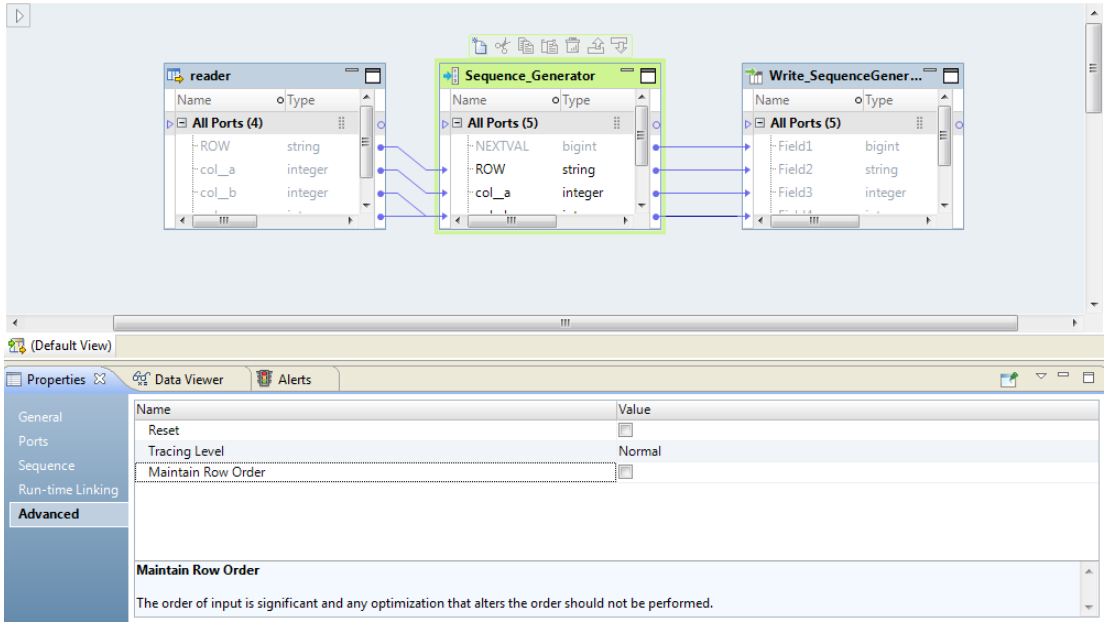
To set a run-time property on a mapping, select the mapping, and then set the property and its value in the **Run-time** tab. Many of the run-time properties outlined in this article are "hidden" properties, meaning you will have to manually enter the property name in the dialog box. Click the edit icon to open the Execution Parameters dialog box, as shown in the following image:



Set the property name and value in the Execution Parameters dialog box, as show in the following image:



To set an advanced property on a transformation, select the transformation, and then select or clear the property, as shown in the following image:



Optimize Caching Transformations

The Data Integration Service stores conditional values and output values in a cache when processing caching transformations such as the Aggregator, Joiner, and Lookup transformations.

If the memory allocated to each cache is not sufficient, the cache data is spilled, or written to disk. Set explicit cache sizes for each Lookup, Joiner, or Aggregator transformation included in a mapping to avoid this issue.

Select a transformation in the Developer tool, and then set numeric values in bytes for the following run-time properties in the **Advanced** tab:

- <Transformation type> Data Cache Size
- <Transformation type> Index Cache Size

Enable Compression for the Distributed Cache

The Data Integration Service can write files to a distributed cache when running a mapping that includes map-side Aggregator, map-side Joiner, or Lookup transformations on the Blaze engine. The distributed cache delivers the cached files to multiple tasklets for processing.

You can enable compression for the distributed cache to improve processing performance. Supported compression libraries include bzip2, gzip, Lempel–Ziv–Oberhumer (LZO), and Snappy. Use Snappy as the compression library for best performance.

If you want to use LZO compression, LZO must be enabled on the Hadoop cluster.

Set one of the values listed in the following property in the `hadoopEnv.properties` file on each Data Integration Service node:

Property	Value
infacal.hadoop.yarn.dc.codec.extension	- .bz2 - .gz - .lzo - .snappy

Disable Map-side Aggregation if a Unique Key is Used as the group by Key in an Aggregator Transformation

In the Blaze engine, map-side aggregation is analogous to the aggregation done in the Map phase in a MapReduce job that runs in the Hive environment. Source data is aggregated based on the group by port set in an Aggregator transformation. The aggregated data then moves to the data shuffle stage for the second level of aggregation.

Map-side aggregation might cause performance overhead if you specify a unique key as the group by port in an Aggregator transformation. Explicitly disable map-side aggregation in the mapping before running the mapping on the Blaze engine if you specify a unique key as the group by port.

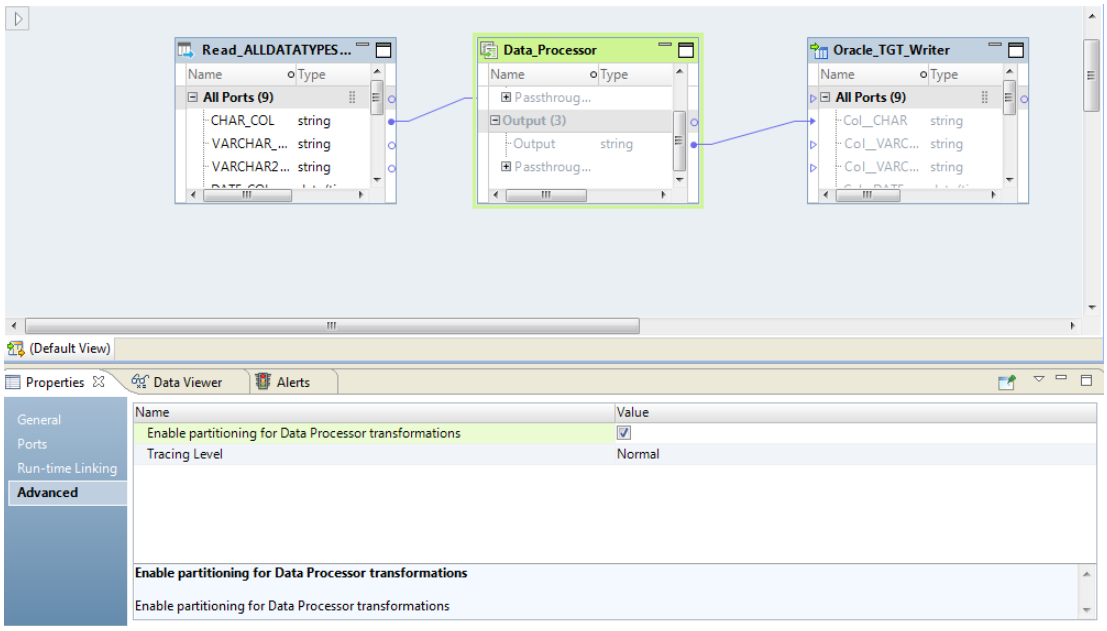
Select a mapping, and then set the following run-time property in the **Run-time** tab in the Developer tool to disable map-side aggregation:

Parameter	Value
GridExecutor.EnableMapSideAgg	false

Enable Partitioning for Data Processor Transformations

Partitioning is extremely useful when processing mappings containing a Data Processor transformation.

You must explicitly enable partitioning for mappings containing a Data Processor transformation. On the Advanced tab of the Data Processor transformation, select **Enable partitioning for Data Processor transformations**, as shown in the following image:



When the Blaze engine runs a Data Processor transformation with multiple output groups, it process the mapping in a single tasklet by default.

Configure the Blaze engine to first stage the data generated by the Data Processor transformation at each output group. The Blaze engine partitions the data and spawns the correct number of tasklets needed to run the mapping logic after processing.

Set the following run-time property on the mapping in the Developer tool to stage data at every output group:

Parameter	Value
Blaze.StageOutputGroupDataForInstances	The name of the Data Processor transformation instance.

Run Mappings with Non-partitioned Relational Database Sources on the Hive Engine

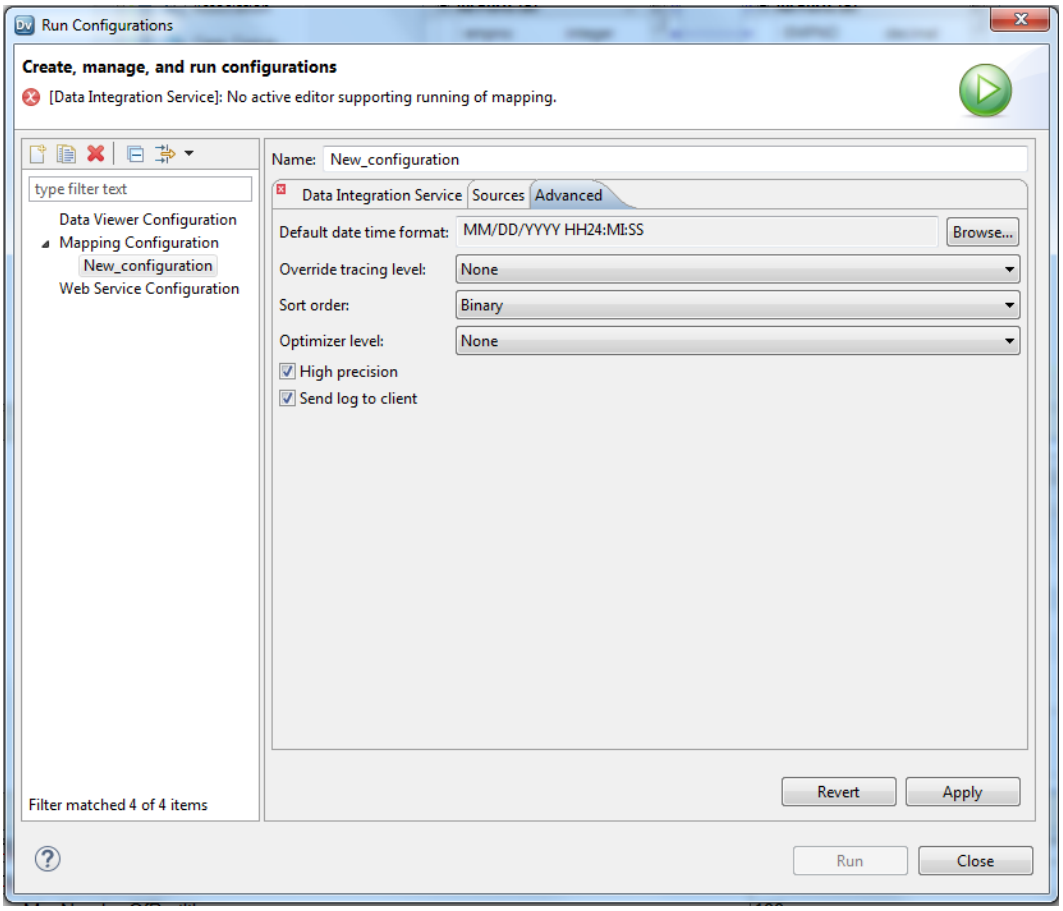
When the Blaze engine runs a mapping that includes a partitioned relational database source, it creates a number of tasklets equal to the number of partitions for optimal performance. However, the Blaze engine spawns a single tasklet to process non-partitioned relational database sources. The result is slower processing.

To optimize performance, run the mapping on the Hive engine instead of on the Blaze engine. The Hive engine uses MapReduce to process the relational database source more efficiently.

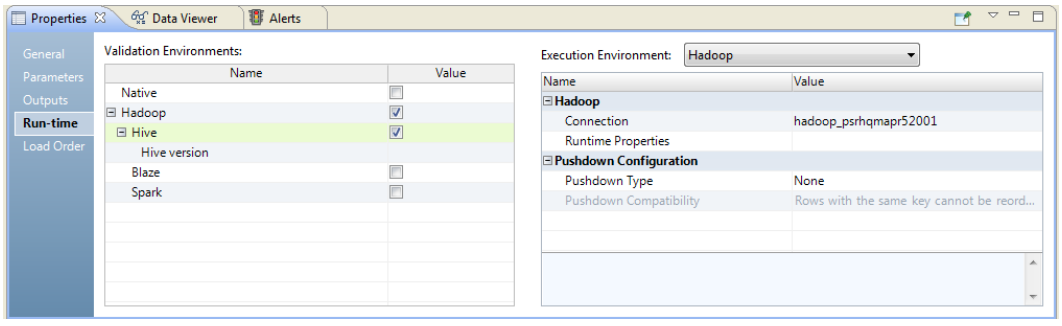
To run the mapping on the Hive engine, perform the following steps:

1. Run the `DBMS_STATS.GATHER_TABLE_STATS` procedure on the source database tables to get the number of rows in the table.

2. Set the primary key on the Physical Data Object representing the source. The primary key determines partitioning. The primary key must either be a Numeric data type (with scale 0) or a String data type.
3. Set the Optimizer level to **None** in the Run Configurations dialog box in the Developer tool to prevent the Data Integration Service from performing optimizations on the source, as shown in the following image:



4. Set the mapping to run in the Hive environment in the Developer tool, as shown in the following image:



5. Set the `RAPartitioning.AutoPartitioningEnabled` run-time property for the mapping to `true` in the Developer tool.

You can also configure the run-time properties in the following table to control partitioning. The values shown in the table are the default values:

Parameter	Default Value
ExecutionContextOptions.Optimizer.AllowInfaReadStagingElimination	false
RAPartitioning.NumberOfPartitions	10
RAPartitioning.MaxNumberOfPartitions	100
RAPartitioning.MinRelationalSourceCardinality	10000000

Set Preferred Partition Size for Partitioned Hive Targets

A mapping that runs on the Blaze engine can have partitioned or bucketed Hive tables as targets. Informatica optimizes the mapping at run-time by adding a Sorter transformation to the mapping just before the Write transformation.

The following image illustrates how Informatica optimizes the mapping by adding a Sorter transformation:



When processing data sources larger than 100 GB, the Blaze engine increases the preferred partition size to 512 MB or 1 GB. This causes the Sorter transformation to run out of memory and spill blocks of data to disk, resulting in performance degradation.

To avoid this issue, explicitly set the preferred partition size to 256 MB and disable automatic partition size adjustment. Set the following run-time properties on the mapping in the Developer tool:

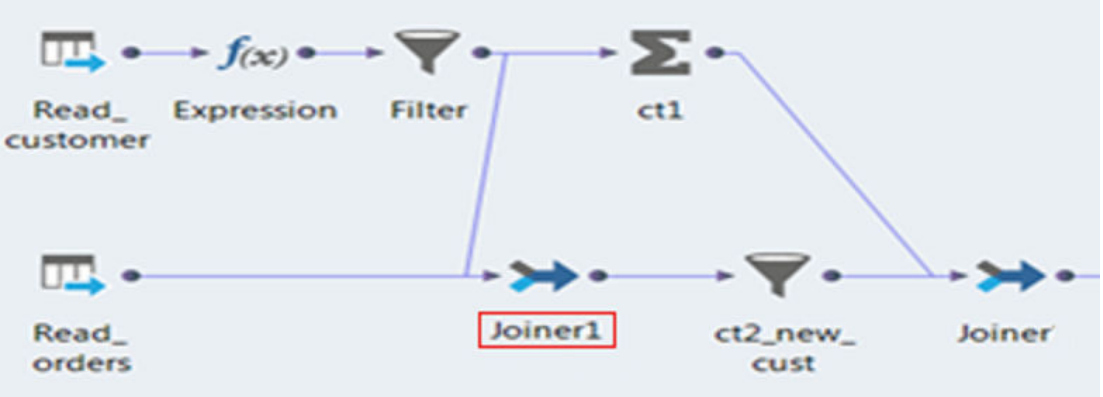
Property	Value
Partitioning.PreferredPartitionSize	268435456
Partitioning.EnableAutoAdjustPartitionSize	false

Disable Map-side Join for Large Data Sources

In a mapping where the master group in a join contains a Filter transformation, the Blaze engine might incorrectly convert the source to a map-side join. The source data is written to the Hadoop distributed cache and copied to all

nodes in the cluster. The result is slower performance in processing data sources, especially for sources larger than 3 GB.

The following image shows a Joiner transformation (Joiner1) that the Blaze engine converts to a map-side join at run-time:



To avoid this potential issue, explicitly disable map join for the affected Joiner transformation. Select the mapping in the Developer tool, and then set the following property in the **Run-time** tab:

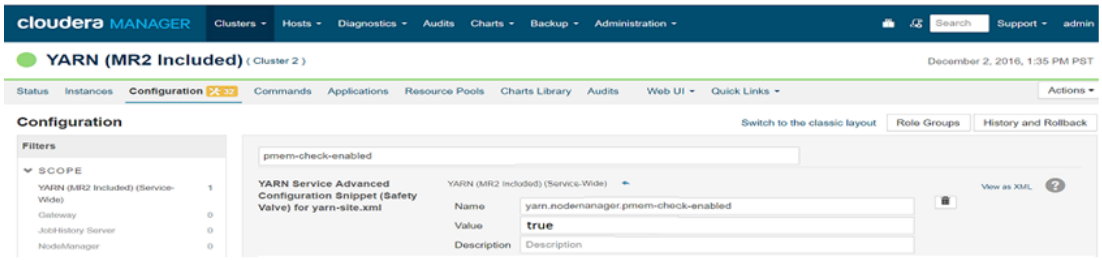
Property	Value
Partitioning.UseMapSideJoin.<Joiner instance name>	false

Disable YARN pmem-check if Using Update Strategy Transformation

When you run a mapping that includes an Update Strategy transformation and writes to a Hive target, the containers the tasklets run in might fail if the data being processed exceeds 10 GB in size.

To avoid this issue, disable the persistent memory check (pmem-check) and restart YARN before running the mapping.

You can use tools such as Ambari or Cloudera Manager to update to disable pmem-check. The following image shows how you can use Cloudera Manager to disable pmem-check:



Author

Dan Hynes